SOCIAL DETECTIVE: THE COMPLETE PLAYTHROUGH & DSA MANUAL

COMPREHENSIVE REFERENCE: EVERY DATA STRUCTURE USED & STEP-BY-STEP SOLUTION WALKTHROUGH

CASE FILE: #001
STATUS: CONVICTED
SCORE: 1000/1000 PTS

CRIME: CORPORATE ESPIONAGE
FACILITY: APEX BIO-INFORMATICS LAB

1. Incident Briefing & Forensic Problem Statement

The Crime: On the evening of October 6, 2026, at precisely **19:42 PM**, the breakthrough CRISPR-Cas9 sequencing file PROJECT_CHIMERA_GENOME.enc was exfiltrated via high-speed USB from Workstation #04 inside Apex Bio-Informatics Lab. The primary database backups were deliberately wiped. Principal Investigator **Dr. Maya Rao** discovered the theft upon returning from the cafeteria at 20:45 PM. Initial access logs show that Dr. Rao's physical keycard (KC-9021) unlocked Door #3 at 19:38 PM. However, physical witnesses and cafeteria camera registers place Dr. Rao on the 1st floor from 19:30 to 20:45 PM. Thirteen suspects had access to campus credentials. The detective must reconstruct the case using **12 custom Python data structures**.

CORE MYSTERY CONTRADICTION:
- Physical Keycard **KC-9021** accessed high-security Door #3 at **19:38 PM**.
- Owner **Dr. Maya Rao** was eating dinner in the cafeteria from **19:30 to 20:45 PM** (Verified by 3 witnesses).
- Therefore, **someone stole Dr. Rao's badge** between 19:30 and 19:38 PM and executed the theft while fabricating their own alibi.

2. Master DSA Architecture — What Data Structure is Used Where

Social Detective does not rely on third-party libraries for its algorithmic logic. All 12 data structures are implemented from scratch in pure Python.

Data Structure	Module Path	Practical Role in Investigation	Time Complexity	Space
Hash Table Separate Chaining	`structures/hash_table.py`	Fast keycard ID, username (e.g. @ryan_47), and clue SHA-256 hash lookup.	Search: **O(1)** avg Insert: **O(1)** avg	O(n + k)
Graph Adjacency List	`structures/graph.py`	Represents 13 suspects (vertices) and 18 communication/credential links (edges).	Add Vertex: **O(1)** Add Edge: **O(1)**	O(V + E)
Breadth-First Search (BFS)	`services/social_network.py`	Finds shortest path (degrees of separation) between Maya and Ryan using FIFO Queue.	Traversal: **O(V + E)** Guarantees shortest hops	O(V)
Depth-First Search (DFS)	`services/social_network.py`	Deep recursive exploration to discover conspirator sub-networks and clusters.	Traversal: **O(V + E)** Via Call Stack	O(V)
Doubly Linked List	`structures/doubly_linked_list.py`	Incident timeline stepper. Supports forward (node.next) and backward (node.prev) traversal.	Next/Prev Step: **O(1)** Full Search: O(n)	O(n)
Singly Linked List	`structures/linked_list.py`	Chain of custody pipeline linking 7 sequential evidence items from theft to exit.	Append: **O(1)** Traverse: O(n)	O(n)
Binary Search Tree (BST)	`structures/bst.py`	Hierarchical index of 15 evidence records by numerical ID (e.g. #1047 CCTV, #1061 USB).	Search: **O(log n)** Worst (skewed): O(n)	O(n)
BST Inorder Traversal	`structures/bst.py`	Recursively traverses Left-Root-Right to output all evidence records in sorted ascending order.	Traversal: **O(n)** Produces sorted list	O(h)
Max-Heap Priority Queue	`structures/heap.py`	Always keeps highest-credibility clue at root (index 0). Bubbles down upon extraction.	Peek Root: **O(1)** Extract Max: **O(log n)**	O(n)
LIFO Stack	`structures/stack.py`	Investigation history ledger. Backtracking ('Undo') pops latest hypothesis in O(1) time.	Push: **O(1)** Pop: **O(1)**	O(n)
Circular Queue Ring Buffer	`structures/circular_queue.py`	Fixed-size (capacity 5) CCTV surveillance monitor buffer. Overwrites oldest frame when full.	Enqueue: **O(1)** Dequeue: **O(1)**	O(K)
Merge Sort & Quick Sort	`structures/sorting.py`	Sorting suspect risk scores (Merge Sort: stable O(n log n); Quick Sort: in-place pivot partitioning).	Avg: **O(n log n)** Live swap trace	O(n) / O(log n)

Binary Search (Array)	`structures/searching.py`	Divide-and-conquer on sorted evidence ID array using low/mid/high boundary pointers.	Search: **O(log n)** Requires sorted list	O(1)

3. Step-by-Step Playthrough Guide (100% Score Solution)

Follow this exact sequence to navigate all 6 investigation stages and secure a conviction against Ryan Mehta.

STAGE 1: Suspect Roster & Alibi Verification ('Who Had Access?')

1. Navigate to Playthrough: Click **'Start Investigation'** or visit /case.
2. Inspect Dr. Maya Rao: Click row #01 (Dr. Maya Rao). Note her statement: she went to the cafeteria at 19:30. Her keycard KC-9021 went missing from her bag. Alibi is confirmed by 3 faculty witnesses. Click **'Tag Suspect' → 'Clear'**.
3. Inspect Ryan Mehta: Click row #02 (Ryan Mehta). Statement: claims he was studying alone in the university library. Has no keycard assigned (KC-NONE). Risk score: **88/100**. Notice: he has NO alibi witnesses. Click **'Tag Suspect' → 'Critical'**.
4. Advance: Click the dominant green banner button: **'INVESTIGATE RYAN MEHTA →'** to advance to Stage 2.

STAGE 2: Social Network Tracing ('Something Doesn't Add Up')

1. Objective: Ryan claimed he barely knew Dr. Maya Rao. We will disprove this using BFS.
2. Execute BFS Trace (Maya → Ryan): Under the graph canvas, verify **Start: Dr. Maya Rao** and **Target: Ryan Mehta**. Click **'TRACE BFS SHORTEST PATH'**.
3. Algorithmic Output: The FIFO Queue visits vertex maya_rao, inspects neighbors, and immediately identifies an edge to ryan_mehta in **1 hop** (Co-authorship on Project Chimera; shared server cluster access). Ryan's claim of having no connection is falsified.
4. Bonus Trace (Ryan → Sam): Click **'Trace Ryan → Sam'**. BFS identifies a 3-hop connection through lab systems logs. Sam Verma is the Linux systems administrator with root privileges.
5. Advance: Click the action banner button **'TRACE CONNECTION →'** to advance to Stage 3.

STAGE 3: Chronological Timeline ('What Happened That Night?')

1. Objective: Traverse the Doubly Linked List forward (node.next) and backward (node.prev).
2. Step Through Events: Click **'NEXT EVENT (node.next)'** repeatedly.
3. Key Contradiction Trigger (19:38 PM): When you land on Event #04 (**19:38 PM: Door #3 Access**), a red alert box triggers: *'ALIBI CONTRADICTION: Keycard KC-9021 scanned into Door #3, but owner Maya Rao was at Cafeteria Station 4.'*
4. Subsequent Events: Advance through **19:42 PM** (Terminal 04 USB dump) and **19:47 PM** (CCTV Camera #1047 captures hooded figure in navy jacket exiting fire stairs).
5. Advance: Click **'FOLLOW THE EVIDENCE →'** to move to Stage 4.

STAGE 4: Forensic Evidence Vault ('Follow the Evidence')

1. Examine Clue Chain (Singly Linked List): Click **'EXAMINE NEXT CLUE IN CHAIN →'**. Observe the pointer advance through Nodes 1 through 7, ending at the NULL pointer.
2. Search BST Record #1047: In the BST Search box, ensure **1047** is entered (or click badge #1047 CCTV). Click **'FIND RECORD →'**. The BST traverses: Root #1075 → Left Child #1047 in **2 comparisons** (O(log n)), retrieving the footage of the hooded figure in the navy jacket.
3. Max-Heap Extraction: Click **'EXTRACT STRONGEST EVIDENCE (O(log n))'**. The Max-Heap extracts Root #1190 (**99% Credibility**: Sandisk USB drive serial number registered to Ryan Mehta's student email). Bubble-down swaps restore the heap invariant.
4. Advance: Click **'BUILD YOUR THEORY →'** to advance to Stage 5.

STAGE 5: Prosecution Theory & Stack Backtracking ('Put the Pieces Together')

1. Suspect Classification: In the suspect tagging roster:
- Click **'Critical'** next to **Ryan Mehta** (marks him as prime culprit).
- Click **'Clear'** next to **Dr. Maya Rao** (verifies her exonerated victim status).
- Click **'Relevant'** next to **Sam Verma** (accomplice proxy).

2. Stack Backtracking Demo: Click **'UNDO (POP)'** to pop the last hypothesis off the LIFO Stack in O(1) time. Re-tag Ryan Mehta as **'Critical'**.
3. Advance: Click **'MAKE FINAL ACCUSATION →'** to proceed to Stage 6.

STAGE 6: Final Accusation & Verdict ('Court of Forensics')

1. Select Perpetrator: In the accusation grid, click on **Ryan Mehta (Research Fellow · Risk: 88%)**. A confirmation card appears outlining the 3 pillars of proof (Stolen Keycard + Fabricated Alibi + USB Drive #1190).
2. Submit Accusation: Click the red **'SUBMIT ACCUSATION'** button.
3. Verdict: The system redirects to /case-solved with victory fanfare: *'100% Case Solved! Ryan Mehta convicted of research espionage ($150,000 crypto payout). Sam Verma cited as accomplice.'*
4. Final Score: 1000 / 1000 PTS (All data structures explored, zero penalties).

4. Academic Viva Voce & Evaluation Cheat Sheet

Be prepared to answer these common oral examination questions during your practical project presentation:

Q1: Why use an Adjacency List instead of an Adjacency Matrix for the social network?

Answer: The social network is *sparse* (13 vertices and 18 edges; E << V^2). An Adjacency Matrix requires O(V^2) space (169 cells) regardless of density. An Adjacency List requires only O(V + E) space and allows BFS/DFS to traverse neighbors in O(V + E) time instead of O(V^2).

Q2: How does the Hash Table handle collisions, and what is its Big-O performance?

Answer: It uses *Separate Chaining* with Singly Linked Lists at each bucket. The hash function computes a polynomial rolling hash modulo table size (default 17). Average time complexity for Search, Insert, and Delete is **O(1)** assuming uniform hashing. Worst-case is O(n) if all keys hash to the same bucket.

Q3: Why is a Doubly Linked List ideal for the incident timeline?

Answer: Detectives must navigate chronologically forward (node.next) to reconstruct events and backward (node.prev) to verify alibis without reallocating arrays. Stepping between adjacent events is strictly **O(1)** pointer manipulation, compared to array slicing or re-indexing.

Q4: What is the difference between BFS and DFS in this investigation?

Answer: BFS uses a FIFO Queue to traverse level-by-level, guaranteeing the *shortest degree of separation* (minimum hops) between Maya and Ryan. DFS uses a LIFO Call Stack to traverse deeply into one branch, which is useful for uncovering cyclic conspiracy chains and sub-group cliques.

Q5: How does the Max-Heap guarantee O(1) peek time for the highest credibility clue?

Answer: A binary Max-Heap maintains the *Heap Invariant* where every parent node has a credibility greater than or equal to its children. Therefore, the maximum element is always at index 0 (root). Extracting the root takes **O(log n)** because the last leaf moves to root and bubbles down.

Q6: Why use a Circular Queue for CCTV monitoring instead of a standard dynamic array?

Answer: CCTV monitors need a bounded ring buffer (e.g. last 5 camera snapshots). A standard array requires O(n) shift operations when removing the oldest element. A Circular Queue uses modulo arithmetic (rear = (rear + 1) % capacity) to achieve strictly **O(1)** enqueue and dequeue with zero memory reallocation.

Q7: What is the time complexity of searching Evidence #1047 in the BST versus Linear Search?

Answer: Linear search scans every element in O(n) (15 comparisons). A balanced BST halves the search space at each step based on numerical comparison, finding Evidence #1047 in O(log_2 15) ≈ **3 to 4 comparisons**.

Q8: What happens during Stack Backtracking (LIFO)?

Answer: Every investigation action (tagging a suspect, dismissing an alibi) is pushed onto the Stack. When the detective clicks 'Undo', the top hypothesis is popped in **O(1)** time, reverting the case board to the exact previous state.